Figure 9.20 The first few steps of nearest neighbor search. The first phase of nearest neighbor search consists of a search on the tree, during which we keep track of the distance of each node (aka point) we traverse: more precisely, if we need to find `N` nearest neighbors for `P`, we need to keep track of the `N` smallest distances we find.
In this case we are showing the query for `N=1`. Thus, if the search is successful, then we have for sure the nearest neighbor, at distance 0. Otherwise, when search ends, we are not yet sure we have found the actual nearest neighbor: in the example above, the point at the minimum distance we have found during traversal is `D` but, as you can see from the figure, we can't be sure that another branch of the tree doesn't have a point within the radius given by `dist(D)`.

However, a k-d tree, much like a sorted array, has structural information about the relative distance and position of its elements, and we can leverage that information to perform our search more efficiently.

How does nearest neighbor search work? We start from the consideration that each tree node covers one of the rectangular regions in which the space was partitioned, as we have shown in figures of 9.5 and 9.6. So first, we want to find which region contains our target point `P`: that's a good starting point for our search, because likely the point stored in the leaf covering that region, `G` in this example, will be among the closest points to `P`.